

Quick Reference Card

Installation & Setup

```
npm install
npm run build
```

Create `.env`:

```
GITHUB_CLIENT_ID=your_id
GITHUB_CLIENT_SECRET=your_secret
OPENAI_API_KEY=your_key # Optional
```

Common Commands

Basic Usage

```
npm start # Run with defaults
node dist/cli.js --help # Show all options
```

Matcher Selection

```
node dist/cli.js --matcher fuzzy # Fast language matching
node dist/cli.js --matcher semantic # AI-powered matching
node dist/cli.js --matcher hybrid # Best results (default)
```

Performance

```
node dist/cli.js --no-deep-analysis # Quick (1-2 min)
node dist/cli.js --max-repos 20 # Limit repos
node dist/cli.js --max-commits 10 # Limit commits
```

Output

```
node dist/cli.js --max-skills 20 # Show more skills
node dist/cli.js --verbose # Detailed logs
node dist/cli.js --output json # JSON format
node dist/cli.js --output-file results.json # Save to file
```

AI Providers

```
# OpenAI (cloud, most accurate)
node dist/cli.js --ai-provider openai --openai-key sk-...

# Google Gemini (cloud)
node dist/cli.js --ai-provider gemini --gemini-key YOUR_KEY

# Ollama (local, privacy-focused)
ollama serve # Start server first
```

```
node dist/cli.js --ai-provider ollama --ollama-model llama3.2
```

```
# No AI (keyword matching)
node dist/cli.js --matcher semantic
```

Flag Reference

Flag	Short	Values	Default
<code>--matcher</code>	<code>-m</code>	fuzzy, semantic, hybrid	hybrid
<code>--max-repos</code>	<code>-r</code>	number	50
<code>--max-commits</code>	<code>-c</code>	number	30
<code>--max-skills</code>	<code>-s</code>	number	15
<code>--deep-analysis</code>	-	boolean	true
<code>--ai-provider</code>	-	openai, gemini, ollama	none
<code>--openai-key</code>	-	string	OPENAI_API_KEY
<code>--gemini-key</code>	-	string	GEMINI_API_KEY
<code>--ollama-url</code>	-	URL	http://localhost:11434
<code>--ollama-model</code>	-	string	llama3.2
<code>--verbose</code>	<code>-v</code>	boolean	false
<code>--output</code>	<code>-o</code>	text, json	text
<code>--output-file</code>	-	path	-
<code>--help</code>	<code>-h</code>	-	-
<code>--version</code>	<code>-V</code>	-	-

Common Scenarios

Quick Check

```
node dist/cli.js --matcher fuzzy --max-repos 20 --no-deep-analysis --max-skills 10
```

Comprehensive Analysis

```
node dist/cli.js --matcher hybrid --max-repos 100 --max-commits 50 --verbose
```

Export JSON

```
node dist/cli.js --output json --output-file analysis.json
```

AI-Powered (Cloud)

```
# OpenAI
node dist/cli.js --matcher semantic --ai-provider openai --openai-key $OPENAI_API_KEY

# Google Gemini
node dist/cli.js --matcher semantic --ai-provider gemini --gemini-key $GEMINI_API_KEY
```

AI-Powered (Local)

```
# Ollama (100% private)
ollama serve
node dist/cli.js --matcher semantic --ai-provider ollama
```

Architecture

```
ISkillsMatcher ← FuzzyMatcher
                ← SemanticMatcher (uses ISemanticProvider)
                ← HybridMatcher
                ← [Your Custom Matcher]

ISemanticProvider ← OpenAIProvider (GPT-4o-mini + embeddings)
                  ← GeminiProvider (Gemini Pro + embeddings)
                  ← OllamaProvider (llama3.2 local + embeddings)
                  ← OllamaProvider (future)
                  ← [Your Custom Provider]
```

Adding Custom Components

New Matcher

```
// src/matchers/MyMatcher.ts
import { ISkillsMatcher } from './ISkillsMatcher';

export class MyMatcher implements ISkillsMatcher {
  name = 'My Matcher';
  async match(profile, skills) {
    return []; // SkillMatch[]
  }
}
```

New AI Provider

```
// src/matchers/providers/MyProvider.ts
import { ISemanticProvider } from '../SemanticMatcher';

export class MyProvider implements ISemanticProvider {
  name = 'My Provider';
  async computeSimilarity(text1, text2) {
    return 0.85; // 0-1
  }
}
```

API Endpoints Used

Github

- `/user` - User info
- `/user/repos` - Repositories
- `/repos/{owner}/{repo}/languages` - Languages
- `/repos/{owner}/{repo}/commits` - Commits
- `/repos/{owner}/{repo}/pulls` - Pull requests
- `/repos/{owner}/{repo}/contributors` - Contributors
- `/repos/{owner}/{repo}/topics` - Topics

Topcoder

- `GET /v5/standardized-skills/skills` - All skills
- `GET /v5/standardized-skills/skills/fuzzymatch?term={term}` - Fuzzy match
- `POST /v5/standardized-skills/skills/semantic-search` - Semantic search

Output Structure

Text

```
🎯 Top N Recommended Skills:
1. Skill Name
   ID: skill-id
   Confidence: 95%
   Evidence: • repo1 • repo2
   Rationale: Why recommended...
```

JSON

```
{
  "user": {"username": "...", "analyzedAt": "..."},
  "stats": {"reposScanned": 50, "commitsAnalyzed": 234, ...},
  "recommendations": [
    {
      "skillId": "...",
      "skillName": "...",
      "confidence": 95,
      "evidence": ["repo1", "repo2"],
      "rationale": "..."
    }
  ]
}
```

Troubleshooting

Rate Limited?

```
# Reduce API calls
node dist/cli.js --max-repos 20 --no-deep-analysis
```

Too Slow?

```
# Skip deep analysis
node dist/cli.js --no-deep-analysis --max-repos 30
```

Missing Skills?

```
# Try different matcher
node dist/cli.js --matcher hybrid --verbose
```

Need Help?

```
node dist/cli.js --help
```

Files & Structure

```
src/
├─ cli.ts           # Main entry point with CLI flags
├─ github.ts        # Github API client
├─ topcoder.ts      # Topcoder API client
├─ ProfileAnalyzer.ts # Deep profile analysis
└─ matchers/
    ├─ ISkillsMatcher.ts # Matcher interface
    ├─ FuzzyMatcher.ts   # Fuzzy matching
    ├─ SemanticMatcher.ts # Semantic matching
    ├─ HybridMatcher.ts  # Combined matching
    └─ providers/
        └─ OpenAIProvider.ts # OpenAI embeddings
```

Key Concepts

Profile Analysis - Scans repos, commits, PRs, languages - Configurable depth (--max-repos, --max-commits) - Respects rate limits automatically

Skill Matching - Pluggable matchers (fuzzy, semantic, hybrid) - Confidence scoring (0-100) - Evidence-based recommendations

AI Integration - Provider-agnostic interface - OpenAI, Ollama, custom providers - Optional (falls back to Topcoder API)